

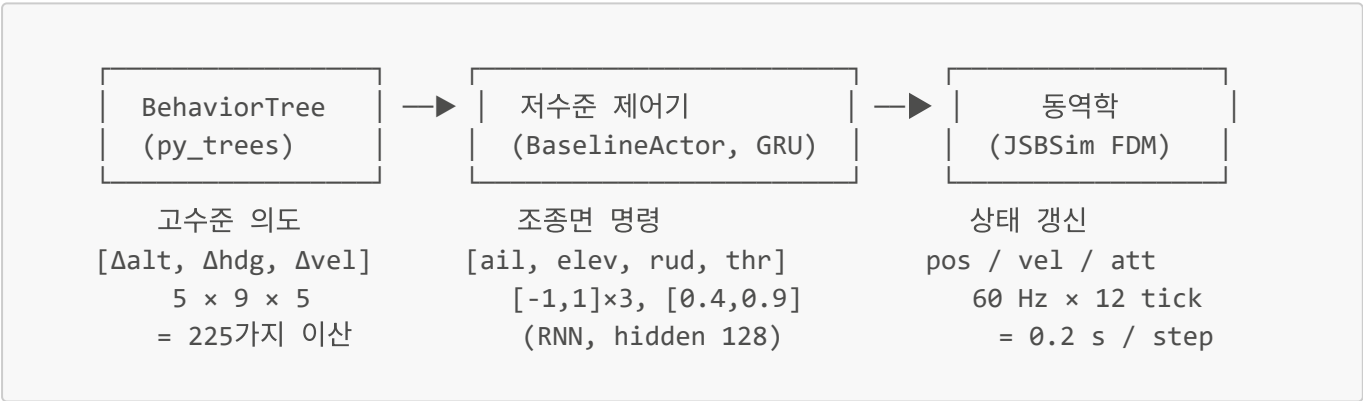
ai-combat 제어기 정리

항공대 경진대회 플랫폼 PPT(항공대경진대회플랫폼.pptx)와 동일한 슬라이드 구성으로 이 프로젝트(ai-combat-core)의 제어 파트를 정리한 문서.

Slide 1. 제어기의 역할

- 본 플랫폼에서도 참가자의 AI 구현 난이도를 낮추기 위해 **저수준 제어기**를 미리 제공한다.
- 항공대 플랫폼이 **기하학 기반 룰**(Forward Vector 정사영 → Alpha/LOS/Beta)로 제어기를 구현한 것과 달리, 이 프로젝트는 **사전 학습된 GRU 기반 RNN**(BaselineActor, model/baseline_model.pt)을 저수준 제어기로 사용한다. → 출처: [Hierarchical Reinforcement Learning for Air-to-Air](#)의 저수준 정책.
- BehaviorTree는 **목적점(VP)**이 아닌 **"고수준 의도"** (Δ고도 / Δ방향 / Δ속도)를 출력한다.
- 저수준 제어기는 그 의도를 받아 **4개의 조종면 명령**(aileron, elevator, rudder, throttle)을 생성하고, JSBSim FDM(F-16 비행역학 모델)이 동역학을 적분한다.
- 성능의 고점을 보려면, 항공대처럼 BT만이 아니라 **저수준 정책(baseline_model.pt)**을 **재학습/교체**하는 것까지 가능하다.

전체 제어 파이프라인



단계	출력 형태	단 위 / 범 위	파일
BT (액션 노드)	[Δalt_idx, Δhdg_idx, Δvel_idx]	이산 5 × 9 × 5 = 225	actions.py
저수준 정책	[ail_idx, elev_idx, rud_idx, thr_idx]	이산 41 × 41	baseline_actor.py

단계	출력 형태	단 위 / 범 위	파일
		×	
		41	
		×	
		30	
JSBSim FCS	fcs/{aileron,elevator,rudder,throttle}- cmd-norm	[-1, 1], 추 력 [0.4, 0.9]	singlecombat_task.py:normalize_action

항공대 플랫폼과의 비교

구분	항공대 경진대회 플랫폼	ai-combat (본 프로젝트)
BT 출력	VP : 목적점 Vector(X,Y,Z)	[Δalt_idx , Δhdg_idx , Δvel_idx] (3축 이산)
제어기 종류	룰 기반 (기하학 정사영)	사전 학습 RNN (GRU 기반 BaselineActor)
제어기 출력	Roll Cmd, Pitch Cmd, Yaw Cmd	aileron, elevator, rudder, throttle
동역학	자체 동역학	JSBSim (F-16 FDM)
수정 자유도	제어기 룰 수정 권장	BT YAML / custom 노드 / <code>baseline_model.pt</code> 교체 가능

Slide 2. 저수준 제어기(BaselineActor) 컨셉 설명

항공대 플랫폼이 "Forward Vector를 법선으로 한 가상 평면에 적기를 정사영시켜 Alpha/LOS/Beta를 구하고 Roll/Pitch/Yaw Cmd를 만든다"는 기하학 컨셉으로 제어기를 정의했다면, 이 프로젝트의 저수준 제어기는 **고수준 의도(Δalt , Δhdg , Δvel)** 와 **자기 상태(ego 9차원)** 를 입력으로 받는 **GRU 기반 시계열 정책**이다.

컨셉 도식

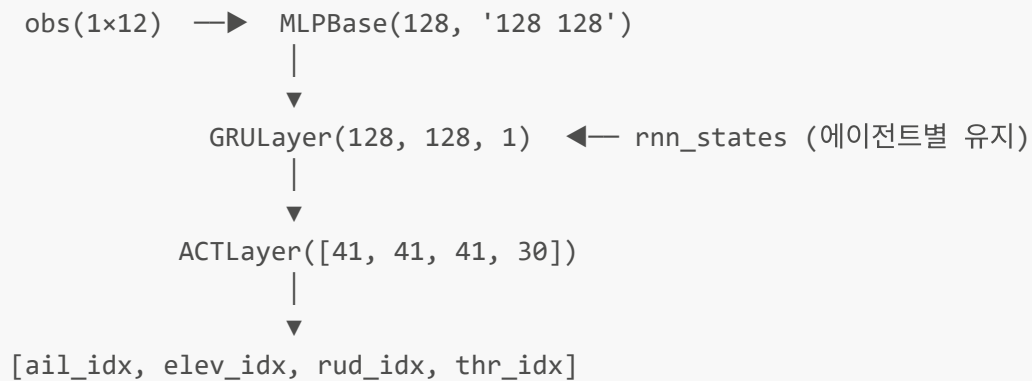
```
HierarchicalSingleCombatTask.normalize_action()

① 고수준 인덱스 → 연속 의도 값
[ $\Delta alt\_idx$ ,  $\Delta hdg\_idx$ ,  $\Delta vel\_idx$ ]
→ norm_delta_altitude / heading / velocity 테이블로 매핑
예: [3, 6, 2] → [+0.15, + $\pi/4$ , 0.0]
```

② 12차원 관측 구성 = 의도 3차원 ⊕ ego 상태 9차원

idx	항목	정규화
0	Δ altitude (의도)	unit 1 km
1	Δ heading (의도)	rad
2	Δ velocity (의도)	Mach
3	ego_altitude	/5000
4	ego_roll_sin	-
5	ego_roll_cos	-
6	ego_pitch_sin	-
7	ego_pitch_cos	-
8	ego_body_v_x	/340
9	ego_body_v_y	/340
10	ego_body_v_z	/340
11	ego_vc	/340

③ BaselineActor (GRU, hidden 128) 추론



④ 이산 인덱스 → 연속 조종면 명령

```

aileron = ail_idx / 20 - 1 → [-1, 1]
elevator = elev_idx / 20 - 1 → [-1, 1]
rudder = rud_idx / 20 - 1 → [-1, 1]
throttle = thr_idx / 58 + 0.4 → [ 0.4, 0.9]

```

```

agent.set_property_values(action_var, norm_act)
JSBSim sim.run() × 12회 (60 Hz × 0.2 s)

```

입력 의도와 출력 명령의 의미 매핑

BT 출력 축	의도(physical meaning)	저수준이 주로 조작하는 채널	항공대 PPT 대응
Δ altitude (5)	상승/유지/하강 의도	elevator 위주 + throttle 보조	Pitch Cmd(LOS)

BT 출력 축	의도(physical meaning)	저수준이 주로 조작하는 채널	항공대 PPT 대응
$\Delta heading$ (9)	좌/우 선회 의도 ($-\pi/2 \sim +\pi/2$)	aileron 위주 + rudder 보조	Roll Cmd (Alpha) + Yaw Cmd (Beta)
$\Delta velocity$ (5)	가/감속 의도	throttle 위주	별도 (항공대는 추력 미포함)

항공대 컨셉 → 본 프로젝트 컨셉으로 읽기

- **항공대 Alpha (Roll Cmd)** ↔ 본 프로젝트 $\Delta heading$ 의 좌우 부호 + magnitude → 두 경우 모두 "적기/목표 방향의 횡방향 편차"가 클수록 큰 롤 입력을 유발한다.
- **항공대 LOS (Pitch Cmd)** ↔ 본 프로젝트 $\Delta altitude$ → 항공대는 정사영 평면에서의 종방향 편차로 직접 산출, 본 프로젝트는 고도 차분을 의도로 주고 RNN이 elevator로 변환.
- **항공대 Beta (Yaw Cmd)** ↔ 본 프로젝트의 rudder 채널 (BT는 직접 지시하지 않고, GRU 정책이 안정화/coordinated turn 용도로 사용).
- **항공대에는 없는 throttle 축**이 추가되어, BT가 에너지 관리(가/감속)를 직접 지시할 수 있다.

성능의 고점을 보려면 (= 항공대 슬라이드 1의 마지막 줄과 같은 메시지)

1. **BT 수준**: `submissions/{name}/nodes/custom_actions.py` 에서 고수준 의도를 더 정교하게 산출 (예: `viper1` 의 `ViperStrike` 가 `tau_deg`, `distance_ft`, `alt_gap_ft` 로 인덱스 산출).
2. **기하학 파라미터 수준**: `src/control/combat_geometry.py` 의 ATA / AA / HCA / TAU / TurnRate / OvershootRisk 를 활용해 항공대 PPT의 Alpha/LOS/Beta 와 유사한 정보를 BT 의사결정 단계에서 사용.
3. **저수준 제어기 자체 교체**: `model/baseline_model.pt` 를 자체 학습한 정책으로 교체하거나, `BaselineAgent` 를 상속한 rule-based 저수준 제어기를 만들어 PPT처럼 **기하학 기반 롤 제어기**로 대체하는 것도 가능.

참고 코드 위치

항공대 PPT 개념	본 프로젝트 대응 위치
BehaviorTree 박스	<code>src/behavior_tree/task.py</code> , <code>src/behavior_tree/nodes/actions.py</code>
VP(목적점) 산출에 쓸 기하학	<code>src/control/combat_geometry.py</code> (ATA/AA/HCA/TAU/TurnRate 등)
제어기 박스	<code>src/simulation/envs/JSBSim/tasks/singlecombat_task.py</code> <code>.normalize_action()</code>
제어기 내부 (Alpha/LOS/Beta 자리)	<code>src/simulation/envs/JSBSim/model/baseline_actor.py</code> (MLP + GRU + ACTLayer)
동역학 박스	<code>src/simulation/envs/JSBSim/core/simulator.py</code> (<code>AircraftSimulator.run()</code> → JSBSim C++)
전체 흐름 상세 분석	<code>bt_to_jsbsim_control_flow.md</code>